Projektuppgift

Programmering i C# (DT071G)

Projektrapport
Utgiftskoll

Pontus Johansson



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Pontus Johansson, pojo2001@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
HT, 2026

Sammanfattning

Många har svårt att hålla koll på sina utgifter under en månad, eftersom utgifter sker vid olika tillfällen och det saknas en samlad överblick. Befintliga budgetappar är ofta omfattande och kräver inloggning eller koppling till bank. Syftet med detta projekt har varit att utveckla en enkel konsolapplikation med hjälp av C# och .NET. Arbetet har genomförts iterativt, där logiken först byggdes och testades innan menyn och en funktion i taget lades till. Lösningen delades upp i fyra stycken delar, en klass för utgifter, en klass för budget logiken och beräkningar, en klass för lagring och ett program som hanterar menyn och all in och utmatning. Beräkningarna görs med LINQ, där datan sparas till en JSON-fil med System.Text.Json så att det finns kvar efter omstart. Validering av delaktiv inmatning hanteras med TryParse så att applikationen inte kraschar. Resultatet är en fungerande konsolapplikation där användaren kan ange en budget, lägga till och ta bort utgifter. Sedan kan användaren se totalt spender, vad som är kvar och vad budgeten är satt till. Samtliga mål har uppfyllts, vilket har verifierats med 17 manuella testfall. Applikationen kan vidareutvecklas med stöd för flera månader, möjlighet att redigera utgifter och ett grafiskt gränssnitt.

Nyckelord: C#, .NET, konsolapplikation, Budget, LINQ, JSON.

Innehåll

Sammanfattning 2
1 Introduktion 4
1.1 Bakgrund och problemmotivering 4
1.2 Övergripande syfte 4
1.3 Avgränsningar 4
1.4 Konkreta och verifierbara mål 4
1.5 Översikt 5
2 Teori 6
2.1 C# 6
2.2 .NET 6
2.3 Objektorienterad programmering 6
2.4 Samlingar och LINQ 6
2.5 JSON 7
2.6 Felhantering 7
3 Metod 8
3.1 Arbetsmetod 8
3.2 Uppdelning av lösning 8
3.3 Verktyg och programvara 8
3.4 Testmetod 8
4 Konstruktion 9
4.1 Övergripande struktur 9
4.2 Gränssnitt 10
4.3 Programflöde 10
4.4 Datamodell 11
4.5 Hantering av inmatning 11
4.6 Beräkningar 13
4.7 Lagring 13
5 Resultat 14
5.1 Måluppfyllelse 14
5.2 Tester 16
6 Slutsatser 17
6.1 Måluppfyllelse 17
6.2 Problem under arbetet 17
6.3 Vad jag hade gjort annorlunda 17
6.4 Vidareutveckling 17
6.5 Vad jag har lärt mig 18

1 Introduktion

1.1 Bakgrund och problemmotivering

Många har svårt att hålla koll på sin ekonomi under en månad. Utgifter för till exempel mat, boende och nöje sker vid olika tillfällen och via olika betalsätt, vilket gör det svårt att veta hur mycket pengar som faktiskt har gått åt och hur mycket som återstår. Utan en samlad överblick är det lätt att överskrida sin budget utan att märka det förrän i slutet av månaden.

Det finns många budgetappar på marknaden, men de är ofta omfattande och kräver till exempel inloggning eller koppling till bankkonton. För den som bara vill ha en enkel överblick över en månads utgifter finns det behov av ett mer lättviktigt verktyg.

1.2 Övergripande syfte

Syftet med projektet är att utveckla en konsolapplikation i C# och .NET som hjälper användaren att hålla koll på en månadsbudget. Användaren ska kunna ange sin budget, registrera och ta bort utgifter, samt se hur mycket som är spenderat och hur mycket som återstår. Projektet syftar även till att tillämpa de kunskaper som erhållits i kursen Programmering i C#.NET.

1.3 Avgränsningar

Applikationen hanterar endast en månad i taget och har ett textbaserat gränssnitt i konsolen. Den har inget grafiskt gränssnitt, ingen inloggning och ingen koppling till bank eller andra externa tjänster. Data sparas lokalt i en fil på användarens dator.

1.4 Konkreta och verifierbara mål

Applikationen ska:

1. låta användaren ange ett budgetbelopp för månaden,
2. låta användaren lägga till en utgift med beskrivning, belopp och kategori,
3. visa en lista över alla registrerade utgifter,
4. låta användaren ta bort en utgift,

5. räkna ut och visa totalt spenderat belopp och återstående budget,
6. visa summan av utgifterna per kategori,
7. spara data till fil så att den finns kvar när programmet startas om,
8. hantera felaktig inmatning utan att krascha,
9. låta användaren ändra budgetbeloppet.

1.5 Översikt

Kapitel 2 beskriver den teori som behövs för att förstå rapporten, bland annat C# och .NET. Kapitel 3 redogör för metoden och de verktyg som har använts. Kapitel 4 beskriver hur applikationen konstruerades. Kapitel 5 presenterar resultatet och genomförda tester, och kapitel 6 innehåller slutsatser och förslag på vidareutveckling.

2 Teori

2.1 C#

C# är ett objektorienterat programmeringsspråk utvecklat av Microsoft. C# är det mest använda programmeringsspråket på .NET. Syntaxen liknar andra språk som Java och JavaScript. C# är starkt typat vilket innebär att varje variabel har en typ som är känd redan när programmet kompileras [1], [2]. Denna typkontroll fångar fel innan programmet körs.

2.2 .NET

.NET är den plattform som C#-program körs på. Plattformen är gratis och fungerar på flera olika operativsystem med en öppen källkod. Plattformen består av en runtime som kör programmet, ett stort standardbiblotek för filhantering och JSON. I .NET SDK ingår utvecklingsverktyg och NuGet är plattformens pakethanterare. Kommandot dotnet används för att skapa, bygga och köra projekt [1], [3].

2.3 Objektorienterad programmering

I objektorienterad programmering byggs programmet upp i klasser som beskriver en sak och vad den kan göra. En klass är som en mall och utifrån denna mall så skapas objekt. I C# används properties för att läsa och ändra data i dessa objekt, och metoder för att utföra handlingar. I detta projekt beskriver klassen Expense en utgift och klassen Budget innehåller en lista med utgifter och metoderna som räknar på dem [1].

2.4 Samlingar och LINQ

För att lagra flera objekt används samlingar. List<T> är en lista som kan växa och Dictionary <Tnyckel, Tvärde> lagrar par av värde och nycklar. LINQ är en uppsättning av funktioner för att ställa frågor mot data direkt i C# syntax [1], [4]. I projektet används LINQ för att summera belopp och gruppera utgifter utifrån kategorier.

2.5 JSON

JSON är ett textformat för att överföra och lagra strukturerad data. Serialisering innebär att man omvandlar ett objekts värde till en form som kan sparas och deserialisering innebär att objektet återskapas från formen. I .NET sköts detta av System.Text.Json, som ingår i ramverket [1], [5].

2.6 Felhantering

Metoden TryParse försöker omvandla en sträng till ett tal och returnerar false om det misslyckas, i stället för att kasta ett undantag. Fel som ändå kan uppstå, till exempel när en fil läses, fångas med try och catch [1].

3 Metod

3.1 Arbetsmetod

Arbetet har genomförts iterativt i små steg. Först skapades klasserna för datan (Expense) och logiken (Budget), och sedan testades med hårdkodade värden innan något användargränssnitt byggdes. Därefter byggdes menyn, och en funktion i taget lades till. Efter varje färdig och testad funktion gjordes en commit i Git, så att det alltid fanns en fungerande version att gå tillbaka till.

3.2 Uppdelning av lösning

Lösningen delades upp i fyra delar med olika ansvar:

- Expense.cs beskriver hur en utgift ser ut.
- Budget.cs innehåller all logik och alla beräkningar.
- BudgetStorage.cs sparar och läser data från fil.
- Program.cs sköter menyn och all in- och utmatning.

Uppdelningen gör att till exempel beräkningarna kan ändras utan att menyn påverkas, och tvärtom.

3.3 Verktyg och programvara

- **Visual Studio Code** med tillägget **C# Dev Kit** som kodeditor
- **.NET SDK 9.0** för att bygga och köra programmet
- **Git** för versionshantering och **GitHub** för lagring av koden
- **Mermaid** för att skapa flödesscheman

3.4 Testmetod

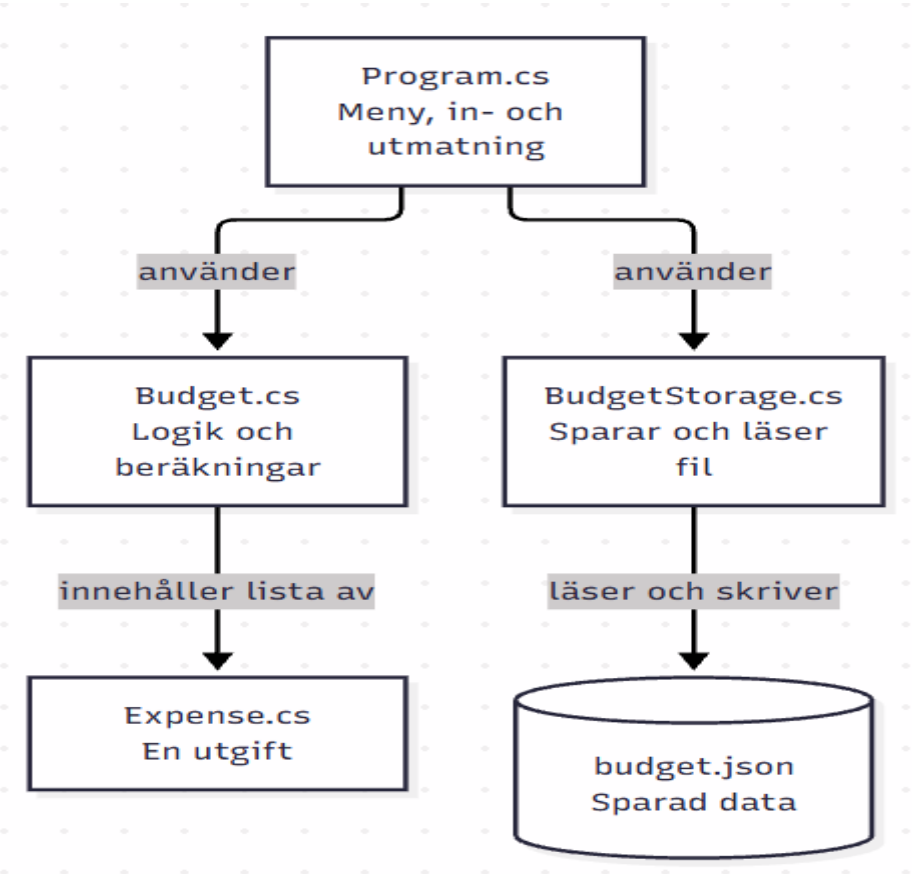
Applikationen testades manuellt genom att varje funktion kördes med både giltig och ogiltig inmatning. Testfallen dokumenterades i en tabell med förväntat och faktiskt resultat, och varje testfall kopplades till ett av målen i avsnitt 1.4. Resultatet redovisas i kapitel 5.

4 Konstruktion

4.1 Övergripande struktur

Applikation är uppbyggd av fyra filer med olika ansvar se figur 1:

- **Expense.cs** - beskriver en utgift med ett id, beskrivning, kategori och belopp.
- **Budget.cs** – innehåller budgetbeloppet, en lista med utgifter och alla beräkningar.
- **BudgetStorage.cs** – sparar budgeten till och läser den från en JSON – fil.
- **Program.cs** – visar menyn och hanterar all in och utmatning.



Figur 1: Applikationens filer och hur de använder varandra.

Uppdelningen gör att inget blandas ihop, Budget vet inget om konsolen och menyn i Program inte räknar ut något själv. Program anropar bara metoder i BudgetStorage och Budget. Alla klasser ligger i namespace Utgiftskoll. Den fullständiga källkoden finns på GitHub [6].

4.2 Gränssnitt

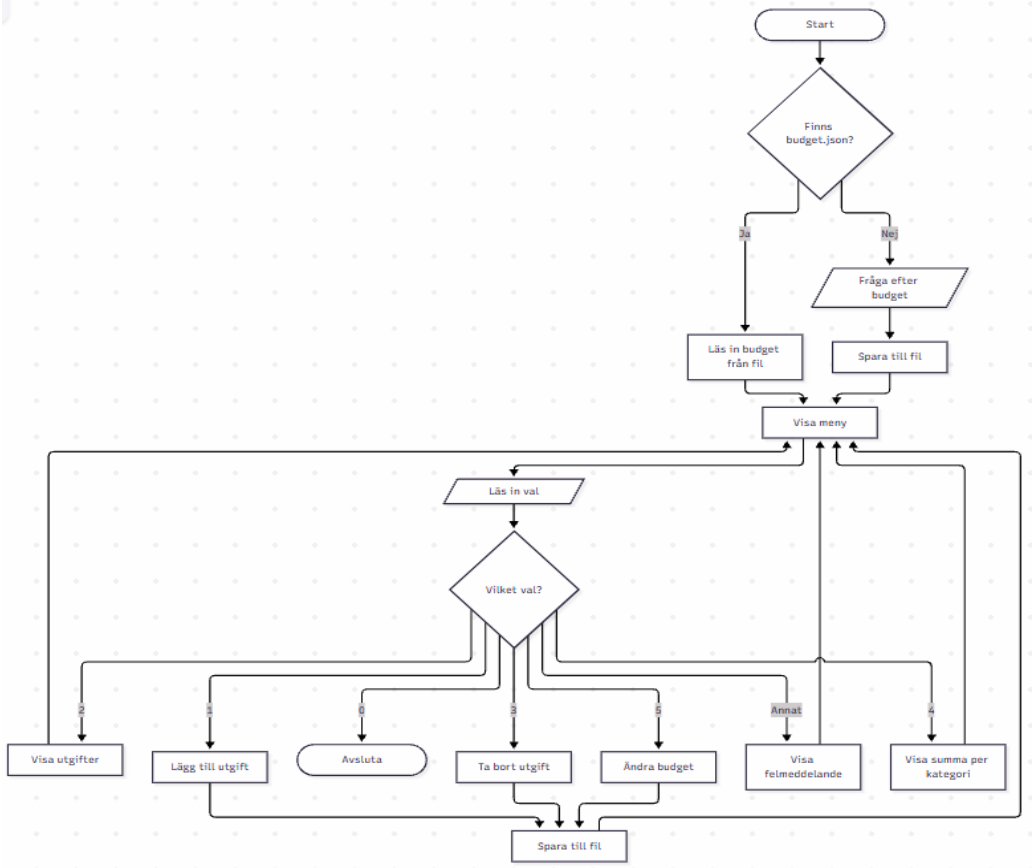
Innan programmeringen skissades tre huvudmenyer, startskärmen, huvudmenyn och vyn för att lägga till en utgift, wireframes visas i figur 2. Eftersom applikationen är en konsolapp består gränssnittet av text. En användare väljer i menyn genom att skriva siffra för de olika valen och trycka Enter. Skärmen rensas med Console.Clear() när en ny visas, detta för att underlätta för användaren.



Figur 2: Wireframes för startskärm, huvudmeny och Lägg till uppgift.

4.3 Programflöde

När programmet startas så försöker programmet läsa in en sparad budget. Om det inte finns någon fil får användaren ange ett budgetbelopp. Därefter visas huvudmenyn med hjälp av en while-loop tills användaren väljer 0 för att avsluta. Valen hanteras med en switch-sats som anropar en funktion för varje menyval. I figur 3 visas detta med hjälp av ett flödesschema.



Figur 3: Flödesschema över programmets huvudflöde.

4.4 Datamodell

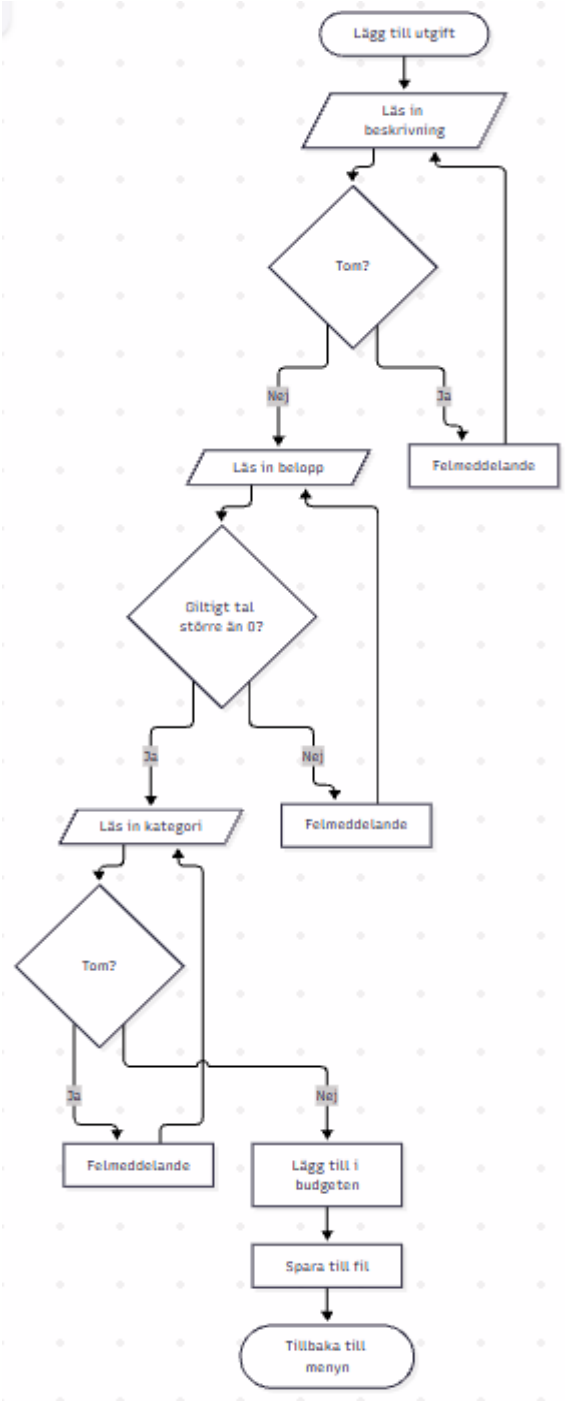
En utgift representeras av klassen Expense. För beloppet används typen decimal i stället för double, då decimal räknar exakt med decimaltal och därför passar bättre eftersom det handlar om pengar.

Varje utgift får ett unikt Id så att användaren kan välja att ta bort en specifik utgift. Id:t delas ut av Budget med hjälp av en räknare NextId, som över med ett för varje ny utgift inlagd. Räknaren sparas tillsammans med alla utgifter, så ett id aldrig återanvänds.

4.5 Hantering av inmatning

All inmatning går genom tre funktioner i Program.cs, ReadText, ReadAmount och ReadId. De frågar om och om tills användaren skriver en giltig input. För belopp används TryParse som returnerar false i stället för att krascha om texten inte är ett tal.

Samma funktion används även när utgift läggs till och när budget anges se figur 4. Kontrollen behöver därför bara skrivas på ett ställe. Programmet följer svenska inställningar så decimaltal skrivs med komma och det skrivs ut tydligt som exempel.



Figur 4: Flödesschema för Lägg till utgift.

4.6 Beräkningar

Alla beräkningar görs i klassen Budget med hjälp av LINQ. Det totala beloppet som spenderats räknas ut genom att summera beloppen och det som återstår är budget minus spenderat.

Summan per kategori räknas ut genom att utgifterna först grupperas på en kategori och summeras sedan per grupp. Resultat blir en Dictionary där nyckeln är kategorin och värdet är summan.

Värdena räknas ut på nytt varje gång de visas i stället för att spara de. Då kan de aldrig bli inaktuellt när en ny utgift läggs till eller tas bort.

4.7 Lagring

Budgeten sparas i filen budget.json med hjälp av System.Text.Json. Hela budgetobjektet sparas med belopp, utgifter och NextId.

Budgeten sparas direkt efter varje ny ändring, i stället för att spara endast när programmet stängs av. Då går ingen data förlorad om användaren stänger fönstret. Vid start läses filen in igen men om filen saknas eller inte går att läsa som JSON fångas detta fel upp med try och catch. Användaren får då ange ny budget och ny fil skapas.

Filen budget.json är tillagd i .gitignore så att användaren uppgifter inte sitter ihop med källkoden.

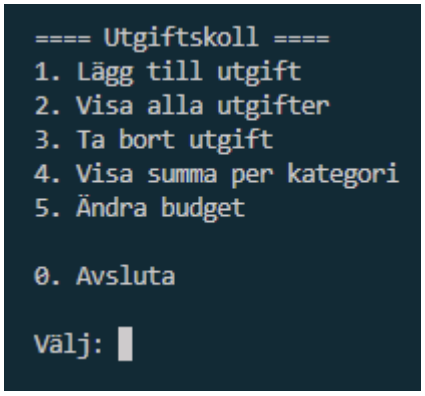
5 Resultat

Resultatet är en fungerande konsolapplikation som uppfyller samtliga mål i avsnitt 1.4. Nedan redovisas det hur varje mål uppfyllts, följt av de tester som genomförts.

5.1 Måluppfyllelse

- **Mål 1: Ange budgetbelopp.**

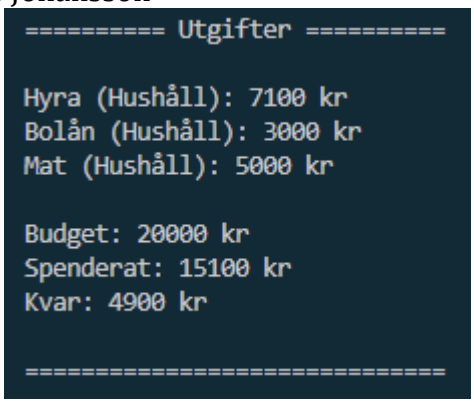
  När man startar applikationen första gången, när ingen sparad fil finns så frågar applikationen efter ett budgetbelopp. Därefter visas huvudmenyn som kan ses i figur 5.



Figur 5: Huvudmeny i den färdiga applikationen.

- **Mål 2-5: Lägga till, visa och ta bort utgifter samt beräkna summor.**

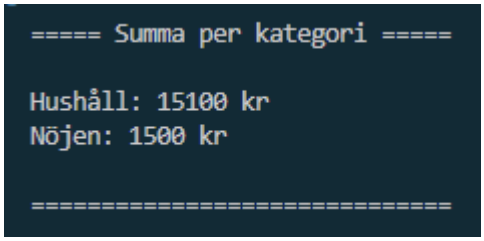
  Användaren kan lägga till en utgift med en beskrivning, belopp och en kategori. Användaren kan även ta bort en utgift genom att ange Id till den utgiften. Under valet ”Visa alla utgifter” listas alla utgifter tillsammans med budget, totalt spenderat och vad som återstår av budget, detta visas i figur 6. Summorna beräknas varje gång en utgift tas bort eller läggs till.



Figur 6: Lista över utgifter med budget, spenderat och kvar.

- **Mål 6: Summa per kategori.**

  Applikationen visar den totala summan för varje kategori som finns sparad. Kategorierna med samma namn men med olika stora eller små bokstäver exempel ”mat” och ”Mat” räknas som samma kategori. Se figur 7.



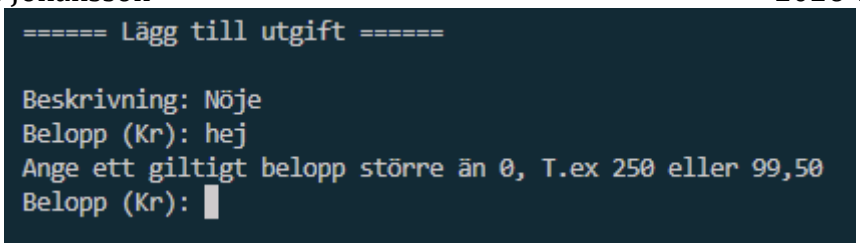
Figur 7: Summa per kategori.

- **Mål 7: Spara data.**

  Budget och utgifterna sparas i filen budget.json efter varje ändring i applikationen. När applikationen startas upp så läses all data från filen om den finns. Om ingen budget.json hittas, så skapas en ny fil och användaren får ange en ny budget.

- **Mål 8: Hantera felaktig inmatning.**

  Felaktig inmatning exempelvis text när man ska ange belopp ger ett felmeddelande och applikationen frågar igen tills att användaren skriver rätt format se detta i figur 8.



Figur 8: Felmeddelande vid ogiltig inmatning.

- **Mål 9: Ändra budgetbelopp.**

  Användaren kan ändra budget med menyval 5 (Figur 5). Det nya beloppet sparas i budget.json. Spenderat belopp och kvarstående räknas då om på nytt.

5.2 Tester

Applikationen testades med 17 testfall, som redovisas i bilaga A. Varje testfall är kopplat till ett eller flera mål i avsnitt 1.4. Samtliga testfall gav förväntat resultat.

6 Slutsatser

6.1 Måluppfyllelse

Syftet med detta projekt var att utveckla en konsolapplikation i C# och .NET som hjälper en användare att skapa en månadsbudget för att ha koll på sina utgifter. Samtliga nio mål har uppfyllts, vilket visas av testerna som gjort och redovisas i kapitel 5. Användaren kan ange en budget och lägga till och ta bort utgifter. Med hjälp av olika val kan användaren se totalt spenderat och belopp som är kvar samt sortera belopp efter kategori. Datan finns kvar efter varje omstart då den sparas ner i en JSON-fil. Vid varje ändring i applikation så sparas datan ner på nytt så att det motverkar krascher. Därmed löser applikationen problem som beskrivs i avsnittet 1.1 att den ger en överblick över en månads utgifter utan inloggning eller koppling till bank.

6.2 Problem under arbetet

De flesta problem var mindre och handlade om skillnader mot de språk jag är van vid, exempelvis TypeScript och JavaScript. Ett exempel är att decimal.TryParse följer datorns språkinställningar, vilket innebär att decimaltal måste skrivas med ett komma i stället för punkt. Under testningen upptäcktes också att några felmeddelande innehöll stavfel, vilket rättades direkt.

6.3 Vad jag hade gjort annorlunda

Arbetssättet genom att bygga upp och testa logiken i Budget innan menyn byggdes fungerade bra, eftersom fel kunde hittas tidigt. Det jag har gjort annorlunda är att skriva testfall redan från start, i stället för när koden var klar. Jag hade också kunnat börja skriva rapporten tidigare parallellt med koden, eftersom rapporten tog längre tid än programmeringen.

6.4 Vidareutveckling

Applikationen kan byggas vidare på flera sätt:

- Flera månader, där varje utgift kan få ett datum, så att användaren kan bläddra mellan månader i stället för att radera filen vid månadsskiftet.
- Redigera utgifter, i dag måste en felaktig utgift tas bort och läggas till igen.

- Varning vid överskriden budget, applikationen kan tydligt visa när det spenderade beloppet överstiger budgeten.

- Grafiskt gränssnitt, till exempel en webbapplikation i .NET Core, eller ett tydligare konsolgränssnitt med biblioteket Spectre.Console.

6.5 Vad jag har lärt mig

Projektet har gett mig en grundläggande förståelse för C# och .NET. Mycket kändes bekant från TypeScript och JavaScript, exempelvis klasser och metoder. Det som var nytt var bland annat typen decimal, TryParse, LINQ-metoder som GroupBy och hur System.Text.Json kan spara ett helt objekt till filen med några rader kod bara. Jag har också fått öva på att dela upp ett program i klasser med tydliga ansvarsområden.

Källförteckning

[1] M. J. Price, *C# 8.0 and .NET Core 3.0 – Modern Cross-Platform Development*. 4 uppl. Birmingham: Packt Publishing, 2019.

[2] Microsoft, "A tour of the C# language", https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/overview Hämtad 2026-10-29.

[3] Microsoft, ".NET introduction and overview", https://learn.microsoft.com/en-us/dotnet/core/introduction Hämtad 2026-10-29.

[4] Microsoft, "Language Integrated Query (LINQ)", https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/statements/linq Hämtad 2026-10-29.

[5] Microsoft, "JSON serialization and deserialization in .NET – overview", https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview Hämtad 2026-10-29.

[6] P. Johansson, "Utgiftskoll", GitHub, https://github.com/Punttt/utgiftskoll Hämtad 2026-10-31.

Bilaga A: Testfall

Redovisning av 17 testfall

Nr	Mål	Test	Förväntat resultat	Faktiskt resultat	OK
1	1	Starta utan sparad fil	Appen frågar efter budget	Appen frågar efter budget	Ja
2	8	Ange "hej" som budget	Felmeddelande, frågar igen	Felmeddelande, frågar igen	Ja
3	8	Ange "-100" som budget	Felmeddelande, frågar igen	Felmeddelande, frågar igen	Ja
4	1	Ange "10000" som budget	Menyn visas	Menyn visas	Ja
5	2	Lägg till: Hyra, 5000, Hushåll	Bekräftelse	Bekräftelse visas	Ja
6	8	Lägg till: tom beskrivning	"Fältet får inte vara tomt"	Felmeddelande, frågar igen	Ja
7	8	Lägg till: belopp "abc"	Felmeddelande, frågar igen	Felmeddelande, frågar igen	Ja
8	2	Lägg till: Mat, 1250,50, hushåll	Bekräftelse	Bekräftelse visas	Ja
9	3, 5	Visa alla utgifter	Spenderat 6250,50, kvar 3749,50	Spenderat 6250,50, kvar 3749,50	Ja
10	6	Summa per kategori	Hushåll: 6250,50 kr	Hushåll: 6250,50 kr	Ja
11	8	Ta bort: ange "hej"	"Ange ett heltal", frågar igen	Felmeddelande, frågar igen	Ja
12	4	Ta bort: ange "99"	"Ingen utgift med id 99"	Meddelande visas	Ja
13	4, 5	Ta bort id 1, visa utgifter	Hyran borta, kvar 8749,50	Hyran borta, kvar 8749,50	Ja
14	9	Ändra budget till 12000	Budget 12000, kvar 10749,50	Budget 12000, kvar 10749,50	Ja
15	7	Avsluta och starta igen	Ingen budgetfråga, data kvar	Ingen budgetfråga, data kvar	Ja
16	8	Ogiltigt menyval "9"	"Ogiltigt val"	Meddelande visas	Ja
17	3	Ta bort alla, visa utgifter	"Inga utgifter inlagda"	Meddelande visas	Ja